



Java Lambda Expressions and Functional Interfaces

Section 1: Learn

What are Lambda Expressions?

Lambda expressions were introduced in **Java 8** to bring **functional programming** capabilities into Java.

They provide a **shorter syntax** for writing **anonymous functions**.

Syntax of Lambda Expression

```
(parameter) -> { statement(s) }
```

Examples:

```
(a, b) -> a + b
```

```
() -> System.out.println("Hello")
```

With types:

```
(String name) -> System.out.println(name)
```

Benefits of Using Lambda Expressions

- **Less boilerplate code**
 - Can be used to **pass behavior** as arguments
 - Great with **Collections and Streams**
 - Makes code **more readable and concise**
-



Lambda Expressions in Action

Sorting a list of names:

```
List<String> names = Arrays.asList("Amit", "Neha", "Ravi");  
names.sort((a, b) -> a.compareTo(b));
```

Using with **Runnable**:

```
Runnable r = () -> System.out.println("Running thread");  
new Thread(r).start();
```

What is a Functional Interface?

A **functional interface** is an interface that has **only one abstract method**.

You can use a **lambda expression** to represent its implementation.

Example:

```
@FunctionalInterface  
interface Greeting {  
    void sayHello();  
}
```

Built-in Functional Interfaces in Java 8

Interface	Description	Example Use
Predicate <T>	Tests a condition, returns boolean	Filtering a list
Function <T,R>	Takes input, returns output	Mapping values
Supplier <	Provides values (no	Lazy



Interface	Description	Example Use
<code>T></code>	input)	initialization
<code>Consume</code> <code>r<T></code>	Takes input, returns nothing	Printing/ logging

Examples

Predicate Example:

```
Predicate<String> isShort = s -> s.length() < 5;  
System.out.println(isShort.test("Hi")); // true
```

Function Example:

```
Function<Integer, Integer> square = x -> x * x;  
System.out.println(square.apply(4)); // 16
```

Supplier Example:

```
Supplier<String> greet = () -> "Hello from supplier!";  
System.out.println(greet.get());
```

Consumer Example:

```
Consumer<String> printer = s -> System.out.println("Hello " + s);  
printer.accept("Java");
```

Creating Custom Functional Interface

```
@FunctionalInterface  
interface Calculator {  
    int operate(int a, int b);  
}
```



```
Calculator add = (a, b) -> a + b;  
System.out.println(add.operate(5, 3)); // 8
```

Section 2: Practice

Sort with Lambda

```
List<Integer> numbers = Arrays.asList(3, 1, 2);  
numbers.sort((a, b) -> b - a);  
System.out.println(numbers); // [3, 2, 1]
```

Filter using Predicate

```
List<String> names = Arrays.asList("Anu", "Neha", "Priya");  
names.stream().filter(n -> n.length() < 5).forEach(System.out::println);
```

Section 3: Know More (FAQs)

Can a functional interface have default or static methods?

Yes. But it can have only **one abstract method**.

Can lambda expressions replace anonymous classes?

Yes, but only for **functional interfaces**.

Are lambda expressions objects?

Yes, they are converted into instances of functional interfaces using the **invokedynamic** bytecode.



Why use functional interfaces?

To pass **behavior as data**, especially in **streams** and **event handling**.

Are lambda expressions faster?

They are concise and efficient but actual performance depends on the JVM optimizations.